MEDICATION SAFETY SIGNAL MONITOR · HAND-IN R01

Foundry from scratch

Version 1.1, 5 November 2026: a primer on Palantir Foundry, taught through one real project

The question this report answers

What is Palantir Foundry, which parts of it did this project use, and where do I click to see each one?

One-line answer

Foundry keeps data, the code that cleans it, a model of real-world things (the Ontology) and the apps people use in one place. We used about a dozen of its parts. Each chapter takes one part, says what it is in one line, shows where it sits in our project, and gives the clicks to find it.

What this is	What this is not
• A primer for an engineer who has never opened Foundry • A one-page primer on drug safety data (chapter 2), so the later reports make sense • The key to the codes (D###, A###, DQ##, O#) and build steps used in every report (chapter 3) • A map of what to click for each part, ready for a live walk-through	• Palantir's official documentation: we explain only the parts we used • A full tour of Foundry (no Pipeline Builder, Contour, Quiver, Slate) • Proof of how Foundry behaves at large scale: our project is a 13-drug pilot on the free Developer Tier • A report on the drug-safety results (see R02 and R07)

How to read this. Read chapter 1 for the whole picture on one page. Chapter 2 teaches the drug-safety words (FAERS, PRR, label, recall) in one page. Chapter 3 is the key to the codes and the evidence labels. Chapters 4 to 13 each cover one part of Foundry in the order data flows: from where things live, through data and code, to the Ontology, the app and its health checks. Each chapter ends with a box that says how we know it works. Chapter 14 is a glossary and the numbers. The data-flow diagram is printed sideways on its own page so it can be read.

Contents	Page
01 · Foundry in one page	3
02 · Drug safety in one page	4
03 · Codes, steps and evidence labels	5
04 · Projects and folders	6
05 · Datasets and transactions	7
06 · Code Repositories and transforms	9
07 · Builds, lineage and branches	11
08 · External sources and egress	13
09 · The Ontology	14
10 · Object Explorer	16
11 · Workshop	17
12 · AIP: models in a pipeline	19
13 · Data Health and monitoring	20
14 · Glossary and numbers	21

01

Foundry in one page

Palantir Foundry is a data platform. In one web application it holds the data, the code that transforms the data, a model of the real-world things the data describes, and the apps people use to act on it. Everything has a version history and a lineage (where each piece came from).

Our project is a pilot for a hospital pharmacy safety officer. Each morning she asks three questions: (1) which drugs have side effects reported that the label does not mention, (2) which drugs have an active recall, and (3) what has changed since yesterday. Her trust rule: every flag must trace back to its source. We watch 13 drugs (8 opioids and 5 GLP-1 diabetes and weight-loss drugs, named in chapter 2) using public data only.

The table maps each Foundry part to what it does for us. Read it top to bottom: it is also the path the data takes.

Foundry part	One line	In our project	Ch.
Project and folders	A shared space with permissions	`Medication Safety Signal Monitor`, 9 folders	04
Dataset	A versioned table (or set of files)	`raw_*`, `clean_*`, `ontology_*`, `monitoring_*`	05
Code Repository + transform	Git repo whose Python functions turn datasets into datasets	`pipelines_raw_ingest`, `pipelines_clean`	06
Build, Data Lineage, branch	Run the code; see the graph; keep changes apart	`master` branch, builds per file	07
Source + egress	A permitted connection to an outside system	4 sources: openFDA, DailyMed, RxNav, FAERS files	08
Ontology	Objects, links and Actions on top of datasets	10 `MSSM` object types, 1 Action	09
Object Explorer	Search and browse objects and their links	Signal → AE Stat → Drug → Label	10
Workshop	Point-and-click app builder	Module **MSSM Safety Officer**, 4 pages	11
AIP	Language models inside pipelines and apps	Claude Sonnet 5.5 reads label sections	12
Data Health	Checks and alerts on datasets	4 checks + our own monitoring	13

Two copies of the code, on purpose

Foundry is the runtime. The GitHub folder `foundry/` is the reviewed copy (a *mirror*) of every file that runs there. A change is made and tested in the mirror, pasted into Foundry, committed and built.

What this means for Dana (the safety officer)

The screen answers her three questions, with four limits she should know on day one:

Question 1 moves only every quarter. Signals change only when a new FAERS quarter arrives. At the baseline gate the newest FAERS data was about 130 days old (monitoring check `faers_age_days`). FAERS moved to a new FDA system (AEMS) in March 2026; how the feed continues is not verified yet (DQ20).

Question 3 card = last 30 days. The Morning brief counts 26 change events in the **last 30 days**, not since yesterday. On page 3 she sets the date range to *since my last review*.

Only the project owner can triage today. The *Triage this signal* form is limited to the owner's account. Giving her role that permission is a production step.

The AI quote is not on her screen. The AI read only 2 oxycodone pilot labels. For the top signal (semaglutide × NAION) it read 0 labels, so *not in label* there comes from the exact-text and synonym checks alone. NAION is a *known positive*, not a discovery: EMA PRAC (6 Jun 2025) and a WHO alert (27 Jun 2025) named it, and our 3 US labels do not mention it (DECISIONS learning 2026-11-05).

Gate: Chapter 1: the reader has the map TESTED

Required evidence: Every Foundry part in this report is tied to a real object in our project.

Result: Met: each row above names a dataset, repository, object type or module that exists OBSERVED.

If it fails: Remove the part from the report rather than describe it in general terms.

02

Drug safety in one page

Foundry is the tool. This page is the subject. Every later report uses these words.

Term	What it means here
FAERS	The FDA Adverse Event Reporting System. Doctors, patients, lawyers and drug companies send **reports** of a suspected side effect (an *adverse event*). Anyone can report; nothing is proven. The FDA publishes them as quarterly files. We use 2026 Q1 and Q2.
Report, case, version	One **case** is one patient's event. Later follow-ups send a new *version* of the same case. We keep only the latest version and apply the FDA's delete lists (D022). We hold 792,346 current cases.
Suspect vs concomitant	Each drug on a report has a role: *primary suspect* (PS), *secondary suspect* (SS), *interacting* (I), or *concomitant* (C: taken at the same time, not suspected). We count a drug only as PS, SS or I (D021).
MedDRA preferred term (PT)	The standard medical word for a reaction, for example *Optic ischaemic neuropathy*. FAERS uses it. MedDRA's groupings need a licence we do not have, so we group terms with our own keyword *buckets* (D016 v3).
Label (SPL)	The drug's official prescribing information, published on **DailyMed** as XML (SPL, Structured Product Labeling). Its safety sections are Boxed Warning, Contraindications, Warnings and Precautions, and Adverse Reactions. Each product has one **reference label** we compare against: the application owner's label for that route (D039 G3).
Recall	An FDA enforcement report: a product taken off the market or fixed. Class I is the most serious (could cause serious harm), Class II may cause temporary harm, Class III is unlikely to cause harm. **Active recall** = openFDA status **Ongoing** (126 of 900). Completed and Terminated recalls are not active.

PRR in one worked line

For each drug and reaction we fill a 2×2 table of reports. Example: semaglutide and NAION (a sudden loss of blood flow to the optic nerve; FAERS term *Optic ischaemic neuropathy*):

	Reaction = NAION	Any other reaction
Semaglutide	a = 394	b = 20,498
All other drugs	c = 128	d = 763,308

PRR (proportional reporting ratio) = (a ÷ (a + b)) ÷ (c ÷ (c + d)) = (394 ÷ 20,892) ÷ (128 ÷ 763,436) = 112.5. In words: 1.9% of semaglutide reports mention NAION, against 0.017% for all other drugs, so NAION is reported about 112.5 times more often with semaglutide. a and a + b are in `numbers.json`; c and d are quoted from diagram 32 v2, which prints the `ontology_ae_stats` row **DERIVED**.

The **Evans rule** (D006, proposed) is the screening test: at least 3 reports (a ≥ 3), PRR at least 2 and chi-squared (a test that the excess is not just noise in the counts) at least 4. NAION passes easily: chi-squared 10,653.8.

Signal and ranked: one meaning each

Word	Meaning (used the same way in every report)
Signal (1,952)	A drug–reaction pair that meets the Evans rule **and** whose reaction is not found in any reference label of that drug (1,844 *not in label*) or is found only through a medium-band synonym (108 *possibly in label*). A reason to look, not proof that the drug causes it.
Ranked (1,073)	The review queue: signals that are *not in label*, in the **clinical** bucket (D016 v3), and less than half carried by mass-polypharmacy reports (reports listing 30 or more drugs, D023 v2). A review queue, not 1,073 findings. It was 1,355 at step 9 (the first full build) and fell as the label check improved.

The 13 drugs. Opioids (8): oxycodone, hydrocodone, morphine, fentanyl, hydromorphone, methadone, buprenorphine, tramadol. GLP-1 drugs (5): semaglutide, liraglutide, dulaglutide, tirzepatide, exenatide. Exenatide has no reference label in our data, so signals exist for 12 of the 13 (D018, D031 v5).

03

Codes, steps and evidence labels

The reports point to project records by short codes. You do not need to open them to follow the text, but every claim can be checked there.

Code	What it is	Where it lives
D### (D006, D016 v3)	A decision: what we chose, why, and the alternative we rejected. A changed decision gets a new version (v2, v3), never an edit. D001 to D048; some are still *proposed*: 12 of them, D006, D015, D025 to D029, D037, D045 to D048	`docs/06-decisions/DECISIONS.md`
A### (A010)	An approach: something we tried, with its result and verdict (keep, drop, park)	`docs/05-approaches/`
DQ## (DQ34)	A data issue found in a public source, and how we handled it	`docs/07-data-quality/data-issues-register.md`
O# (O1 to O3a)	An optimization: before and after numbers, and whether it helped	`docs/08-optimization-log/optimization-log.md`

Code	What it is	Where it lives
G1 to G6	Checks inside decision D039 (for example G3: one reference label per product)	D039
Step N (1 to 12)	A step of our build order. Step 6 = first Ontology path; step 8 = first signal and app screen; step 9 = all 13 drugs (first full build); step 10 = incremental FAERS; step 11 = the three-question screen and monitoring. A step is done when its *gate* (a written check) passes	`docs/04-architecture/foundry-conventions/project-build-order-v1.md`
Baseline gate	The full rebuild on 5 November 2026 whose counts were frozen into `numbers.json`	`docs/11-handin/baseline-gate.md`

Every claim carries a label. They mean the same thing in every report:

Label	Meaning
OBSERVED	Read directly from a public source (FAERS, DailyMed, openFDA, RxNorm), with the query that produced it.
DERIVED	Computed by our pipeline from observed data, for example a PRR or a count per drug and reaction.
ASSUMED	Not verified: taken as true for now because the data or the brief is silent. It could be wrong.
DESIGNED	A design choice we made and wrote down (the decision log says why). It may be built; the status band says how far.
TESTED	Built, run, and checked against evidence.
PLANNED	On the plan; not built yet.
PROPOSED	Suggested; waiting for a decision.

04

Projects and folders

What it is. A Foundry *project* is a shared space with its own permissions. Inside it, *folders* hold every resource: datasets, code repositories, Ontology types, apps. A resource keeps a permanent ID called an **RID** (resource identifier, for example `ri.foundry.main.dataset.…`), so moving or renaming it does not break anything that uses it ASSUMED.

We work in one project on the free **Developer Tier** (D010) OBSERVED. The folders follow the path the data takes DESIGNED:

In our project	Exact name or path
Project	`/kestrel-01f4ab/Medication Safety Signal Monitor`
Data	`Data/raw` (as received), `Data/clean` (checked, one meaning per row), `Data/curated` (backs the Ontology), `Data/quarantine`, plus `Data/monitoring`
Code	`Pipelines/` holds both code repositories
Ontology	`Ontology/Objects`, `Ontology/Links`, `Ontology/Actions`
App	`Workshop/` holds the module MSSM Safety Officer
Other	`AIP/{Logic,Evals}`, `Analysis`, `Documentation`, `Monitoring`, `Scratch`

Three layers carry the data. People call them *bronze, silver, gold* (the *medallion* pattern). We keep the plainer names **raw, clean, curated** DESIGNED (D036):

Layer	Rule	Example
raw (bronze)	Exactly what the source sent, plus when and how we asked	`raw_openfda_enforcement`
clean (silver)	Typed, checked, deduplicated; drug identity resolved once here	`clean_drug_identity`
curated (gold)	One row per real-world object; backs the Ontology	`ontology_signal_signals`

Names follow `<layer>_<source>_<thing>`, for example `clean_faers_cases_current`. The Ontology types all start with `MSSM`, because the Developer Tier Ontology is shared with Palantir's example types DESIGNED.

What to click

1. Left bar → **Projects** (folder icon) → *Medication Safety Signal Monitor*.
2. Open `Data` → `curated` to see the datasets the app reads.
3. Any resource: the breadcrumb at the top (Medication Safe… › Data › curated › name) shows where it lives.

> **How we know**
> The folder layout and paths are recorded in `foundry/README.md` ("Foundry resource map"; 9 top-level folders) and visible in the breadcrumb of every screenshot in this report.

05

Datasets and transactions

What it is. A *dataset* is a table (or a folder of files) with a full version history. Each time something writes to it, Foundry records a **transaction** ASSUMED. A *snapshot* transaction replaces all the rows. An *append* transaction adds rows. Update and delete transactions change files in place. You can look at any past version.

How we use the transaction types OBSERVED:

Pattern	Our dataset	Why
Snapshot every build	All `raw_*` API pulls, e.g. `raw_openfda_enforcement` (930 rows: one per recall per drug it names, so 900 distinct recalls; foundry/README)	Each build is a full, comparable copy of what the source said
Files, not a table	`raw_faers_quarter_files` (FAERS quarterly ZIP members)	FAERS comes as text files; a manifest dataset records their sizes and checksums
Append, kept forever	`monitoring_checks` (21 rows per build)	History of every health check, so a drop can be compared with last time
Read own previous version	`raw_dailymed_spl_xml`, `ai_label_section_extractions`, `ontology_change_snapshot`	Reuse work already done (chapter 6, *incremental*)
Empty on purpose	`ontology_triage_decisions`	Its rows come only from the officer's Action. Action edits are stored in the Ontology, not in this dataset, so it stays empty while 2 decisions exist (chapter 9)

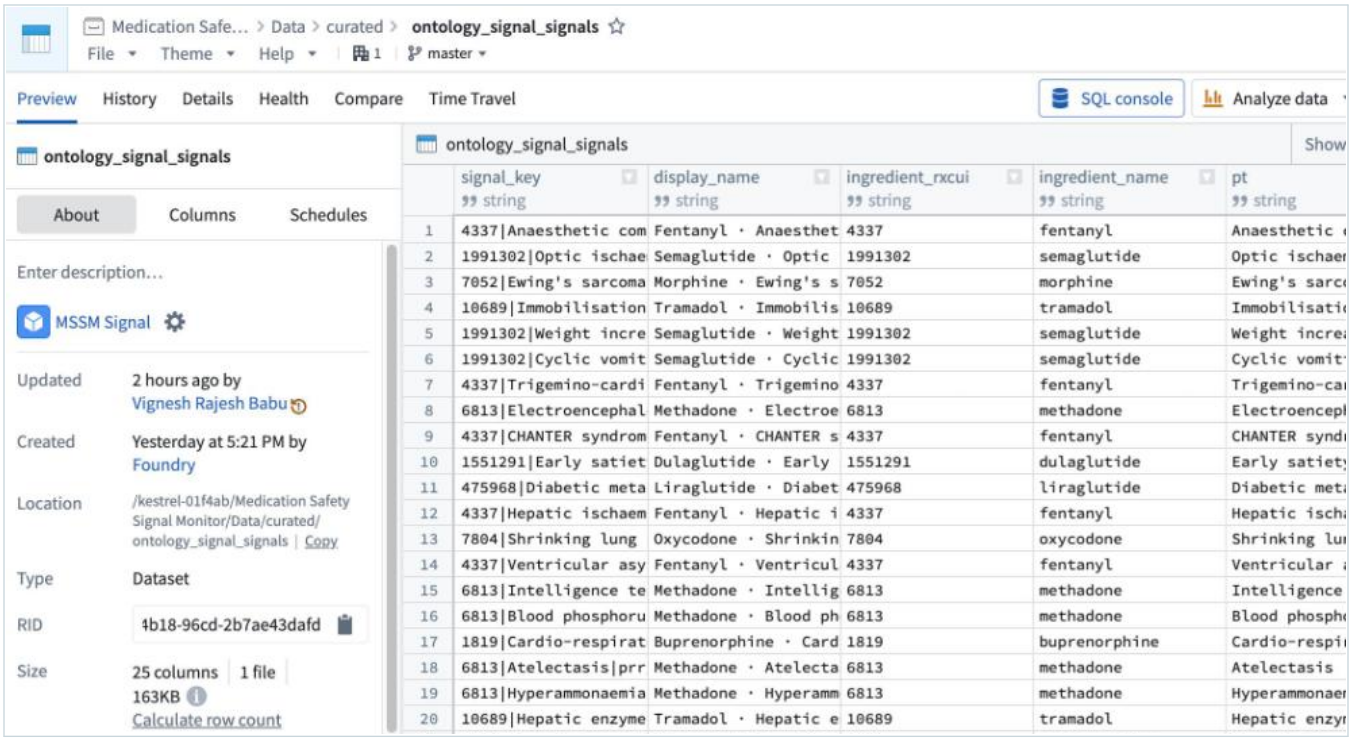


Figure 1. The dataset page for `ontology_signal_signals` (cropped): Preview tab with the first rows, the About panel (location, RID, size, *Updated via pipelines_clean*), and tabs for History, Health and Time Travel.
Source: docs/11-handin/screens/hood-dataset-signals-preview.jpg (Foundry, 2026-11-05)

What to click

1. Open a dataset → **Preview** shows rows; **Columns** shows the schema.
2. **SQL console** (bottom) runs a query on this dataset. It answered in about 1.5 s for us; the code repository's SQL panel once gave no answer in 90 s (DECISIONS learnings, 2026-11-04) OBSERVED.
3. **History** lists every transaction and the build that wrote it. **Time Travel** shows an older version.

> **How we know**
>
> Every count in `numbers.json` was read with SQL in the code repository's SQL panel at the baseline gate (`docs/11-handin/baseline-gate.md`). Raw row counts were compared with each source's own total, drug by drug, after every pull.

06

Code Repositories and transforms

What it is. A *Code Repository* is a git repository that lives inside Foundry, with a web editor, checks and a build button. A **transform** is a Python function that reads one or more datasets (*Inputs*) and writes one or more datasets (*Outputs*). Foundry runs it on its own compute.

In our project	Exact name or path
Ingest repository	`Pipelines/pipelines_raw_ingest`: Spark transforms that call outside APIs; library `transforms-external-systems`
Clean + curated repository	`Pipelines/pipelines_clean`: lightweight polars transforms; datasets in, datasets out, no outside calls
Files	`transforms-python/src/myproject/datasets/` (e.g. `clean_faers.py`, `ontology_signals.py`)
Mirror in GitHub	`foundry/pipelines/{raw,clean,ontology,ai,monitoring}/` (same file names)

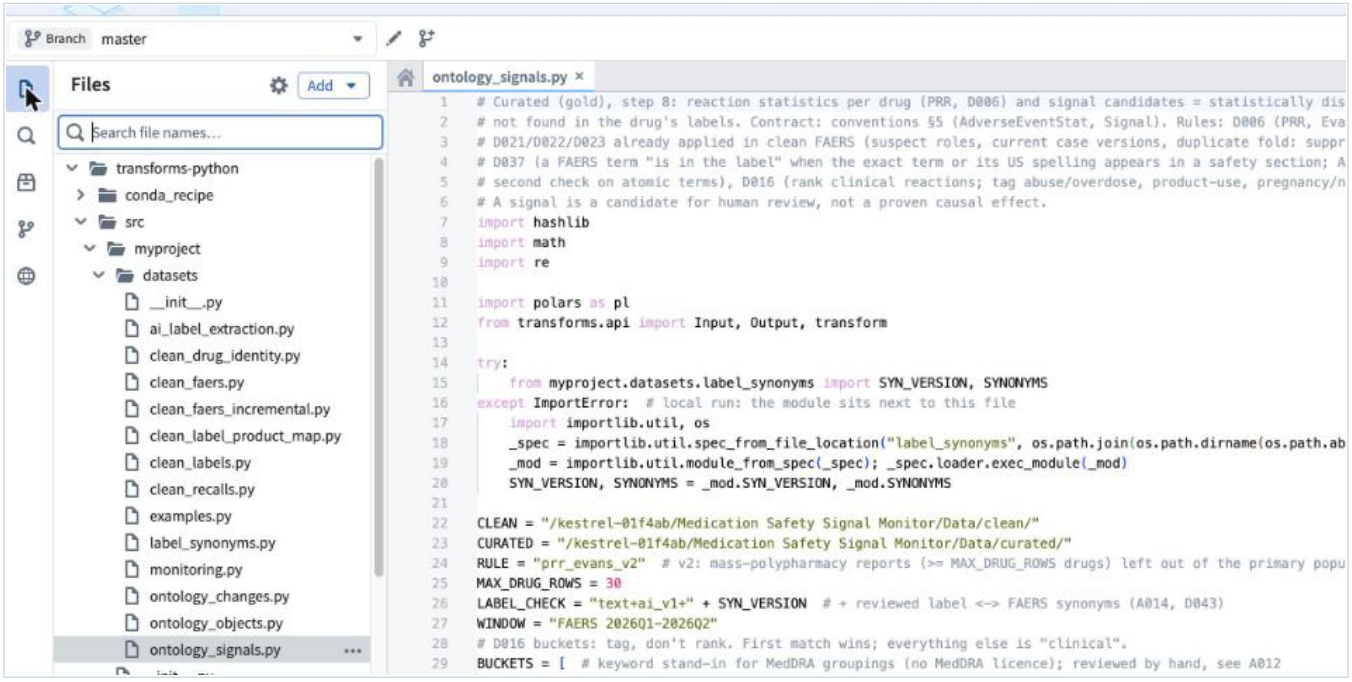


Figure 2. Code Repository `pipelines_clean` on branch `master` (cropped), file `ontology_signals.py` open. Left: the files; right: the code. Commit and Build sit further right on the same bar; the SQL scratchpad is below.
Source: docs/11-handin/screens/hood-code-repo-ontology-signals.jpg (Foundry, 2026-11-05)

Three kinds of transform we used

Kind	What it means	Where we use it
Spark (`@transform`, `@transform_df`)	Runs on a Spark cluster; good for very large data	Raw pulls that call APIs (they need `@external_systems`)

Kind	What it means	Where we use it
Lightweight (`@transform.using`, polars)	Runs on one machine with the polars library; starts fast	Every clean and curated transform. Builds took about 1 minute vs about 4 for Spark, most likely start-up (not measured separately; an observation, not a like-for-like test; DECISIONS learnings, 2026-11-04) OBSERVED
Incremental (`@incremental`)	The transform can read its own previous output and only do new work	AI cache, DailyMed XML reuse, incremental FAERS, change feed, monitoring history

The shape of every clean transform is the same. A shortened example:

```
from transforms.api import Input, Output, transform

@transform.using(out=Output(CLEAN + "clean_recalls_current"),
                 enforcement=Input(RAW + "raw_openfda_enforcement"))
def compute(out, enforcement):
    df = enforcement.polars()          # read the input as a polars table
    ...                                # clean it
    out.write_table(df)                # one output transaction
```

Watch out

Incremental is not automatically faster. Our incremental FAERS path gives exactly the same rows as the full rebuild, but at two quarters it took 88 s against 32 s OBSERVED (O2). Reading back and rewriting the saved state costs more than it saves on a short history. R02 and R06 explain this.

What to click

1. **Pipelines** → `pipelines_clean` → **Code** tab → `transforms-python/src/myproject/datasets/`.
2. Edit, then **Commit** (top right). This is Foundry's own git, not GitHub.
3. **Build** builds the file you have open. **Branches** shows the branch list (we use `master`).

How we know

Both repositories run the code that is mirrored in `foundry/pipelines/`. The local runner (`foundry/tools/local_run`) runs the same lightweight files on a laptop and gave the same counts as Foundry (A014), so a change can be tested before it is pasted in.

07

Builds, lineage and branches

What it is. A **build** runs a transform and writes its outputs as new transactions. **Data Lineage** is Foundry's graph of which dataset feeds which. A **branch** is a separate line of change, as in git; a **commit** saves the code on that branch. Figure 3 (next page, sideways) is our data flow; Data Lineage shows the same graph live.

How a change reaches Foundry

1. Edit the mirror file in GitHub `foundry/` and test it with the local runner.
2. In the Foundry repository, open the file with the same name and paste the new text. Paste, do not type: the editor's auto-indent breaks typed Python.
3. **Commit**, then **Build**.
4. Check the result with SQL and compare the row count with the source total.
5. Record the counts in the project log.

Build habits we learned the hard way

What happened	What we do now
The repository Build button skipped an unchanged incremental transform, even after its input was rebuilt	Build from the dataset page; check its **History** for a new job OBSERVED
A dataset whose code and inputs had not changed did not rebuild	Build ▾ → **Force build**
Data Lineage *build all ancestors* re-ran every raw pull and the 700 MB FAERS download (DECISIONS learnings, 2026-11-04)	Build file by file, in dependency order
A new repository opened a VS Code workspace that auto-committed for about 30 minutes and conflicted with edits (DECISIONS learnings, 2026-11-04)	One editor per branch; commit right after each paste

Branches. We worked on one branch, `master` in the code repositories (Ontology Manager shows its branch as *Main*, top right of Figure 4 in chapter 9) OBSERVED. A team would make a branch per change and merge after review ASSUMED. For this pilot, review happens in GitHub on the mirror instead DESIGNED.

What to click

1. On any dataset page: **Explore pipeline** (or *Explore data lineage*) opens Data Lineage.
2. In the graph, select a node to see its build history; colours show stale or failed builds.
3. Dataset page **Build** ▾ → **Force** → *Only this dataset* rebuilds one step.

> **How we know**
> The baseline gate rebuilt the whole chain from the DailyMed pull to monitoring, in order, and every count matched the earlier values. The change feed found 0 changes between the two builds, which is right because the data was the same.

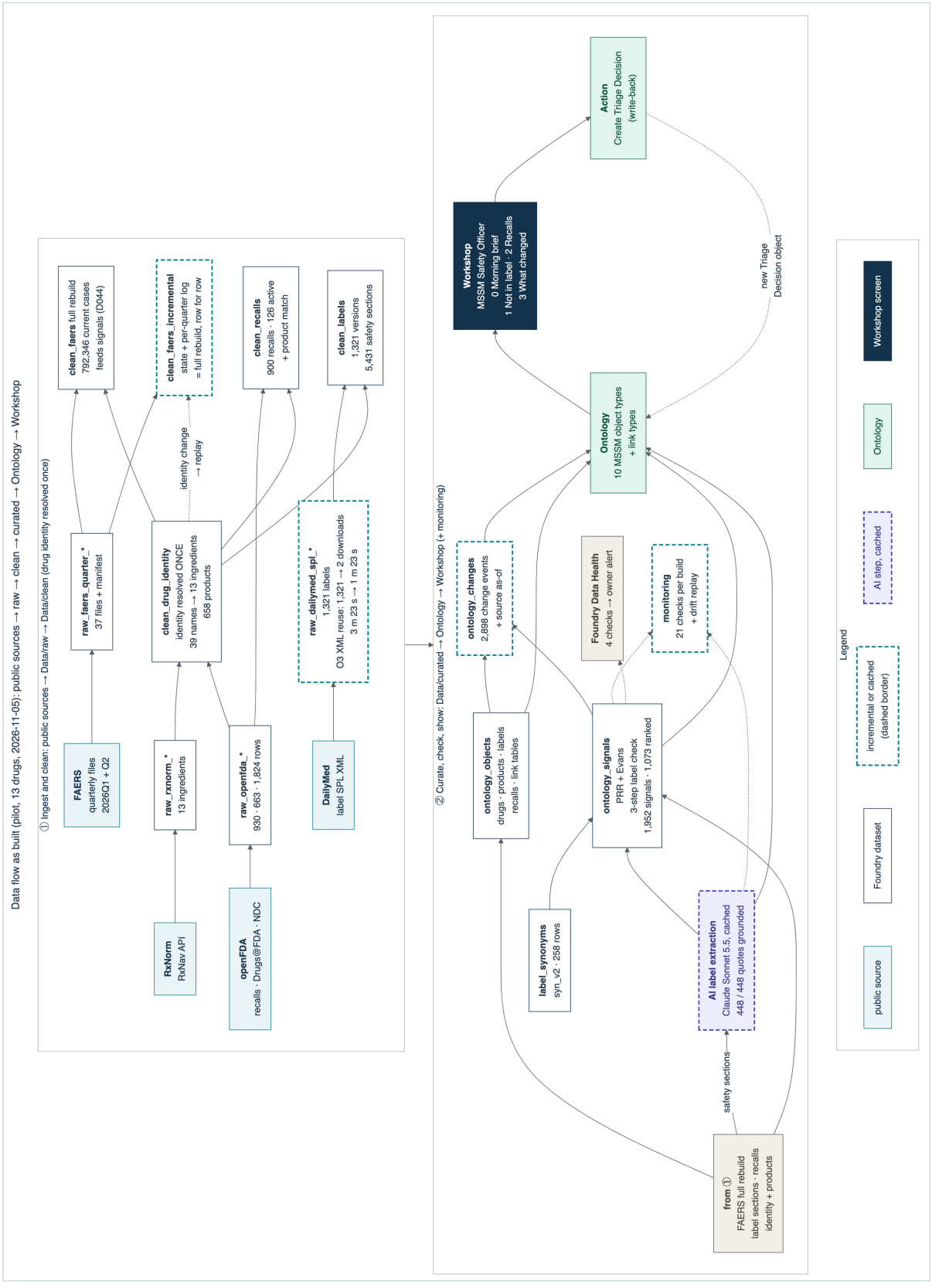


Figure 3. Our data flow as built (v2, O3 measured): 4 public sources → `Data/raw` → `Data/clean` (drug identity resolved once in `clean_drug_identity`) → `Data/curated` → Ontology → Workshop. Dashed borders are incremental or cached steps. The product count printed in the identity box comes from the README, not from `numbers.json`. (Printed sideways on its own page so the labels can be read.)

Source: docs/diagrams/png/30-handin-data-flow-v2.png (Mermaid source docs/diagrams/src/30-handin-data-flow-v2.mmd)

08

External sources and egress

What it is. Foundry does not let code call the internet freely. A **source** (in *Data Connection*) describes one outside system. An **egress policy** allows network traffic to one host and port. Code can only use a source that has been imported into its repository.

Source	API name	Host (egress policy)	Brings in
`src_openfda_api`	`OpenFdaApi`	api.fda.gov:443	Recalls, Drugs@FDA applications, NDC packages
`src_dailymed_api`	`DailyMedApi`	dailymed.nlm.nih.gov:443	Drug labels (SPL XML)
`src_rxnav_api`	`RxNavApi`	rxnav.nlm.nih.gov:443	RxNorm drug identity
`src_faers_files`	`FaersFiles`	fis.fda.gov:443	FAERS quarterly ZIP files

All four are REST API sources in `Data/raw` with no login. Settings: exports **off**; *import into code repositories* **on**; compute modules and pipeline functions **off** (least privilege, D038). The source secret `OpenFdaApiKey` is not used yet, because the pull needs only about 50 requests, well under the no-key daily limit DESIGNED.

A raw transform uses the source like this (shortened):

```
@external_systems(openfdaapi=Source("ri.magritte..source.e46a…"))
@transform_df(Output(".../Data/raw/raw_openfda_enforcement"))
def compute(openfdaapi):
    ...  # paged search; every reply kept with its HTTP status and time
```

Rule every raw transform follows DESIGNED	Why
Keep the original reply (JSON, XML or file) next to a few parsed columns	Every flag can be traced to what the source said
Save request parameters, HTTP status, retrieval time and the source's own 'last updated'	Freshness and audit
A non-200 reply becomes an **error row**, never a silent gap	A missing label must not look like 'not in label'
Row count must equal the source's own total, per drug	Caught two silent search bugs (R07)

What to click

1. **Data Connection** → *Sources* → `src_openfda_api` → Overview, Network egress, Code import.
2. **Control Panel** → egress policies: check the policy shows *Approved*. The source page kept saying *Pending approval* after self-approval OBSERVED.
3. In the repository: left bar → *Sources* → import the source; reload if it shows an error.

> **How we know**
>
> The first access test (D038) wrote one real recall, D-0338-2026, with HTTP 200 into `raw_openfda_enforcement_access_test`. Today the 13-drug pulls give 900 distinct recalls, 663 Drugs@FDA rows, 1,824 NDC rows and 1,321 DailyMed labels, each equal to the source total.

09

The Ontology

What it is. The **Ontology** is a layer of real-world *things* on top of datasets. An **object type** (for example Drug) is backed by one dataset: one row becomes one *object*, one column becomes one **property**. A **link type** connects two object types (a Drug has many Products). An **Action** is a form that writes to the Ontology, so a person's decision is saved next to the data. Diagram 31 (printed sideways as Figure 2 in R03) shows the whole Ontology; R03 explains it in depth.

Object type	Backing dataset (`Data/curated`)	Primary key	Objects
MSSM Drug	`ontology_drug_drugs`	`ingredient_rxcui`	13
MSSM Product	`ontology_product_products`	`product_key`	not captured at the gate
MSSM Label	`ontology_label_labels`	`label_key`	1,321
MSSM Recall	`ontology_recall_recalls`	`recall_number`	900
MSSM Labeled Reaction	`ontology_labeled_reactions` (from the AI step)	`labeled_reaction_key`	448
MSSM AE Stat	`ontology_ae_stats`	`stat_key`	14,921
MSSM Signal	`ontology_signal_signals`	`signal_key`	1,952
MSSM Change Event	`ontology_change_events`	`event_id`	2,898
MSSM Source Status	`ontology_source_status`	`source`	4 (one per source)
MSSM Triage Decision	`ontology_triage_decisions` (empty; edits on)	`decision_id`	2

Links are built in two ways, by design DESIGNED. The many-to-many links (Drug ↔ Product, Product ↔ Label, Recall ↔ Product, Recall ↔ Drug) are backed by their own join datasets, such as `ontology_link_drug_product`. Many-to-one links use a foreign-key column, such as `ingredient_rxcui` on Signal pointing at Drug.

The one Action: Create MSSM Triage Decision

The officer fills in **Decision** (reviewing, dismissed, escalated or monitoring) and a **Rationale**. Subject type, subject key and **evaluation id** (a fingerprint of the exact computation she is looking at: count, PRR, label check, data window) come from the selected signal and are locked. The system sets decision id, reviewer and time. The Action can only *create*, never edit or delete, so the decisions form an append-only audit trail DESIGNED (D040). Only the project owner can submit it today.

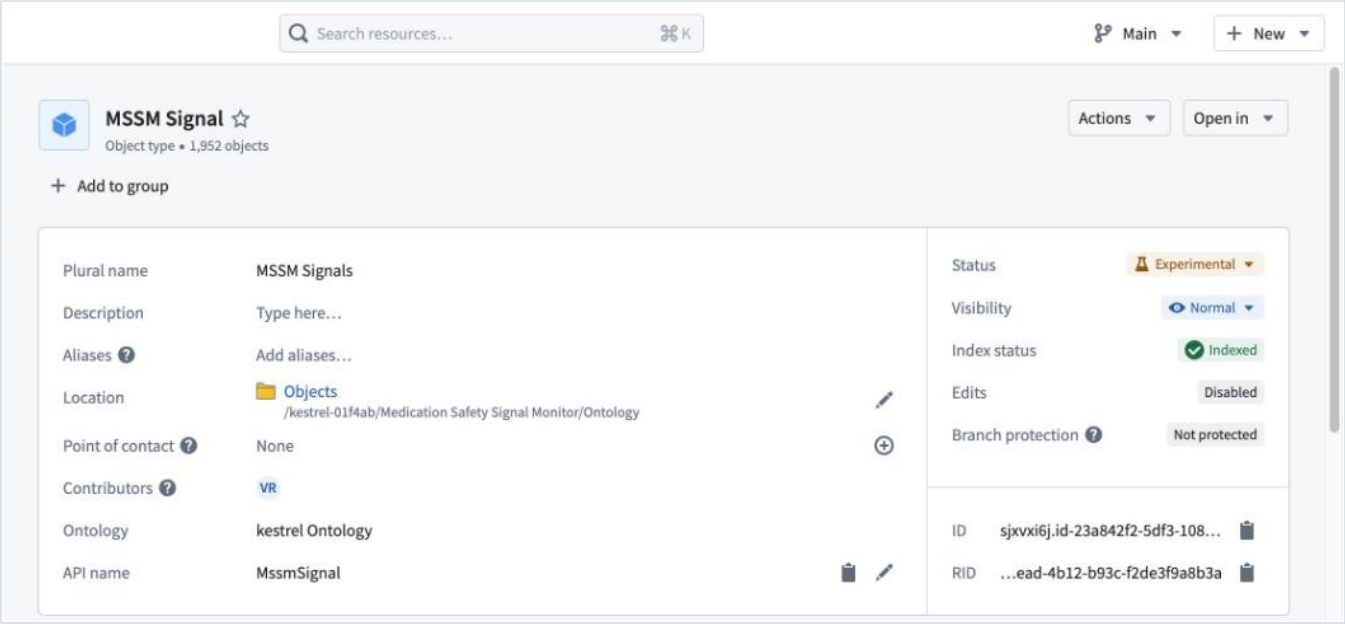


Figure 4. Ontology Manager, object type MSSM Signal (cropped): 1,952 objects, 25 properties, status Experimental, index status Indexed, edits disabled (only Triage Decision allows edits). Top right: the branch selector shows *Main*.
Source: docs/11-handin/screens/hood-ontology-manager-signal.jpg (Foundry, 2026-11-05)

A mistake left visible

A second Signal → Drug link type was created by mistake on 2026-11-05, before checking the record. Nothing uses it. Deleting it is the owner's call, so it is still there and is marked on diagram 31 (R03).

What to click

1. **Ontology Manager** → search `MSSM` → open an object type → *Properties*, *Datasources*, *Link types*.
2. Number display (decimals) is set once on the property (*Display* → *Numeric formatting*); every app uses it.
3. Action types → *Create MSSM Triage Decision*. Its form preview here is a **test run only** and saves nothing.

How we know

Object counts equal the backing dataset rows (checked at the baseline gate).

A test decision survived a forced rebuild of its empty backing dataset and a page reload: Action edits live in the Ontology, not in the dataset (D040). 2 decisions exist today.

10

Object Explorer

What it is. *Object Explorer* is Foundry's search and browse tool for Ontology objects. You search by title or property, open an object, and follow its links. It is the fastest way to check the trust rule by hand.



Figure 5. Object Explorer (cropped): search for *Semaglutide · Optic ischaemic neuropathy*, the MSSM Signal selected, Links tab open: Drug of signal (the duplicate link), MSSM AE Stat, MSSM Drug.
Source: docs/11-handin/screens/hood-object-explorer-naion-links.jpg (Foundry, 2026-11-05)

The trace path we use OBSERVED:

1. Search the signal name, e.g. *Semaglutide · Optic ischaemic neuropathy* (NAION).
2. **Links → MSSM AE Stat**: the 2×2 counts behind the statistic (chapter 2: a = 394 reports with this drug and this reaction; PRR 112.5).
3. **Links → MSSM Drug → MSSM Products → Reference Label**: the exact label versions that were searched, with DailyMed links. The app's page 1 now lists these too (panel *Reference labels searched*: 3 labels for NAION).
4. The FAERS case ids behind `a` can be listed with SQL on `clean_faers_drug_reaction`.

What to click

1. Left bar → **Search** (or Object Explorer) → type the object title.
2. Filter by object type on the left; open the object; tabs **Details** and **Links**.
3. To submit a real triage decision outside the app: open MSSM Triage Decision → **Actions** → Create.

> **How we know**
> At the baseline gate, 5 random ranked signals were traced this way. For all 5, the count `a` recounted from FAERS rows equalled the signal, and the reaction term was not in any safety section of the reference labels searched (`baseline-gate.md`).

11

Workshop

What it is. *Workshop* builds apps on the Ontology without front-end code. A **module** (an app) has **pages**. Each page holds **widgets** (tables, filters, charts, buttons). **Variables** carry data between widgets, for example *the set of ranked signals* or *the signal the user selected*. **Events** say what happens on a click, such as *switch to page 2*.

In our project	Exact name or path
Module	**MSSM Safety Officer** (header *MSSM · Morning safety review*), folder `Workshop/`, published v0.9.0
Pages	0 · Morning brief · 1 · Not in label · 2 · Recalls · 3 · What changed (a Tabs widget switches them)

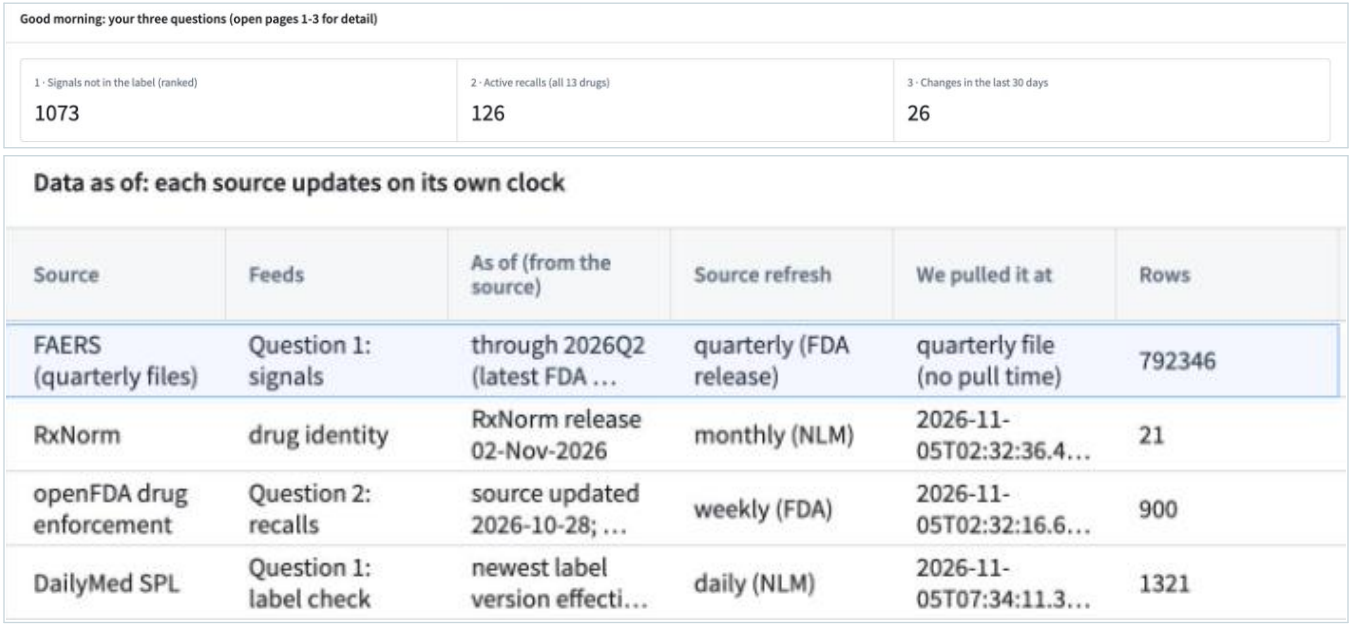


Figure 6. Page 0 · Morning brief (two crops): one Metric card with three numbers (1,073 ranked signals, 126 active recalls, 26 changes in the **last 30 days**), and below it the table of how fresh each source is (*Data as of*).
Source: docs/11-handin/screens/app-0-morning-brief.jpg (Foundry, 2026-11-05)

Workshop idea	Our example
Widget: Metric card	Three counts on page 0, each an *Object set aggregation* (Count) variable
Widget: Object table	*Ranked signals (not in label)* on page 1; recalls on page 2; changes on page 3
Widget: Filter list	Drug filter (pills with counts per drug); Event Date histogram with start and end date
Widget: Property list	The full signal record: 22 of the 25 signal properties in reading order with plain labels (foundry/ontology/README.md)
Widget: Button group	*Triage this signal* runs the Action
Variable: object set	*Ranked signals* = MSSM Signal where Is Ranked = true
Variable: filtered object set	*Ranked signals for chosen drugs* = *Ranked signals* + the drug filter

Workshop idea	Our example
Variable: object property	*Selected signal key*, *Selected evaluation id* from the table's active row
Event	Each tab's on-click = *Switch to page*

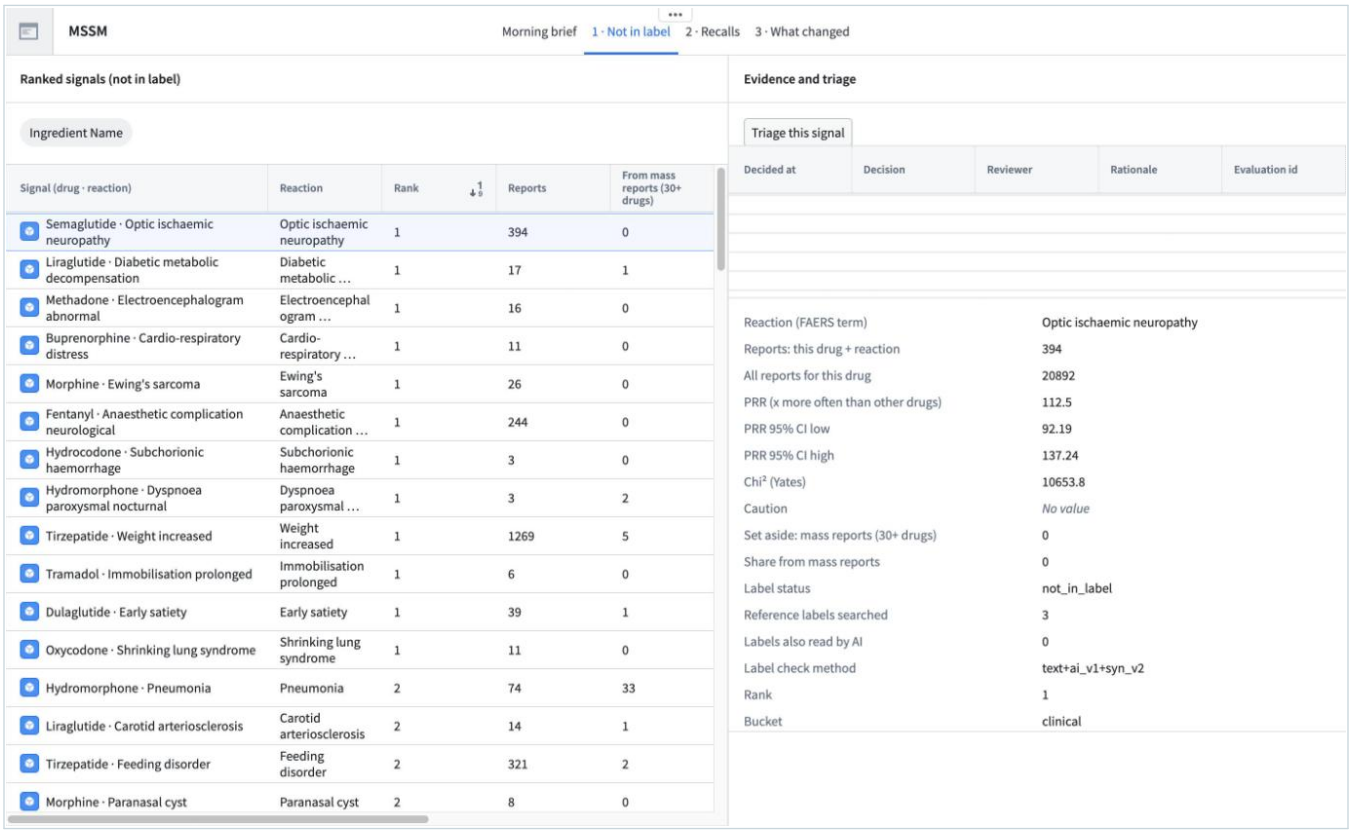


Figure 7. Page 1 · Not in label: ranked signals on the left (semaglutide × NAION at rank 1, 394 reports), the Triage button, past decisions, and the full signal record on the right.
Source: docs/11-handin/screens/app-1-not-in-label-naion.jpg (Foundry, 2026-11-05)

> **Note**
>
> **Why two variables per page.** A filter list must read the *unfiltered* set. If it read the filtered set, picking one drug would remove every other drug from its own choices. So: unfiltered set → filter list → filtered set → table → selected row → detail panel. Pages 2 and 3 use the same pattern.
>
> **A shortcut.** Table columns and property-list fields can be pasted into the widget's JSON (</> → edit → *Validate and save*). Reordering and relabelling the 22 properties took one paste instead of about 60 clicks (DECISIONS learnings, 2026-11-04) OBSERVED. The JSON box is live: close it without saving if you typed into it by mistake.

What to click

1. **Workshop** folder → *MSSM Safety Officer* → **View** to use it, **Edit** to change it.
2. In Edit: the left panel lists pages and widgets; the right panel shows the selected widget's data source (a variable) and its events.
3. Name pages with a number (0 ·, 1 ·): the editor sorts them alphabetically.

How we know

In View mode: select a signal → *Triage this signal* → the form opens with subject and evaluation id filled and locked → Submit → reload → the decision is listed under the signal (D042). The three Morning brief numbers equal SQL counts in `numbers.json`.

12

AIP: models in a pipeline

What it is. AIP (Palantir's Artificial Intelligence Platform) gives Foundry access to large language models (LLMs) from several vendors through a *Model Catalog*. Models can be used in apps, in *AIP Logic* functions, or, as we did, inside a Code Repository transform.

Step	What we did OBSERVED
Access test	An AIP Logic test replied in 2.9 s; Claude, Gemini and Grok models were listed (`docs/10-delivery/2026-11-04-foundry-access-results.md`)
Pick a model	Claude Sonnet 5.5 (`ri.language-model-service..language-model.anthropic-claude-5-5-sonnet`), D041
Import it into the project	Without this the build fails with `AccessWithoutImportDenied`. The Import button appears only when the model RID is typed inline in `GenericCompletionLanguageModelInput("…")`
Call it from a transform	`ai_label_extraction.py` in `pipelines_clean`: one call per label safety section
Check it	Code checks that every quote the model returns is found word for word in the section text
Cache it	`@incremental`: the cache key is label + version + section + text hash + prompt, parser and model version

Results on the pilot (2 oxycodone labels, 9 sections): 448 reactions extracted, 448 quotes found word for word in all 9 sections. A rebuild with unchanged text made 0 model calls (9 of 9 sections from cache) OBSERVED.

Accuracy is a separate, smaller test. F1 (a score that balances missed and wrong items) was 0.97 against the proposed D015 target of 0.85, on a 35-item answer key: the Boxed Warning and Contraindications sections of the 2 oxycodone labels. The model found 35 of 35 answer-key items DERIVED (`eval/ai-eval-v1.md`).

What this does not show

The answer key was written by Claude and frozen before the model ran; a pharmacist has not checked it yet.

Grounding shows the model did not invent quotes. It does not show the item is the right one.

Today the AI does not change the signal list: text matching already covers its terms for the 2 pilot labels. Its quote is not shown on the officer's screen yet, and it read no semaglutide label.

What to click

1. **AIP** / Model Catalog: browse available models.
2. Dataset `Data/curated/ai_label_section_extractions`: one row per section with status, tokens, latency and `from_cache`.
3. Dataset `ontology_labeled_reactions` (object type MSSM Labeled Reaction): each reaction with its exact quote.

How we know

Optimization log O1: run 1 made 9 model calls (70,776 tokens); the rebuild made 0 with 0 tokens. AI evaluation: `eval/ai-eval-v1.md`.

13

Data Health and monitoring

What it is. *Data Health* is Foundry's built-in checking on a dataset: did the last build succeed, is a key unique, is the data fresh. A failing check notifies its subscribers. We add our own computed checks for things Foundry cannot know, such as whether the AI's quotes are grounded.

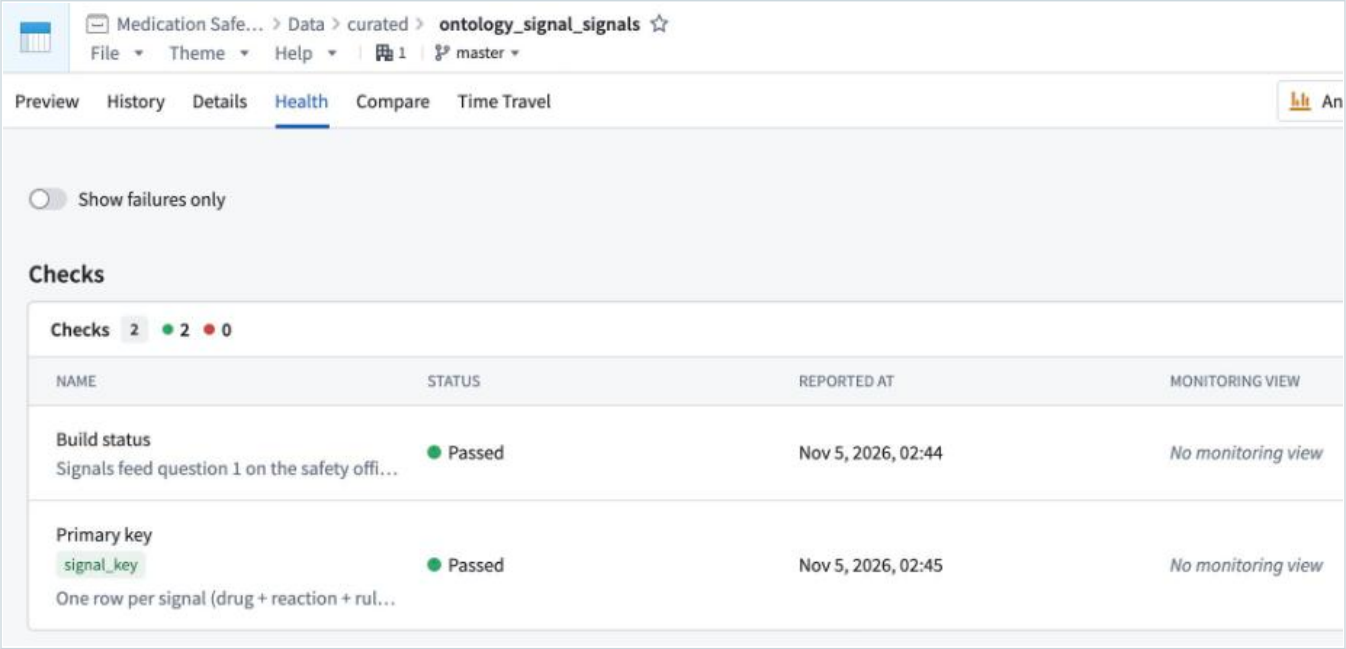


Figure 8. Health tab of `ontology_signal_signals` (cropped): two checks (Build status, Primary key on `signal_key`), both Passed on 5 November 2026.

Source: docs/11-handin/screens/hood-data-health-signals.jpg (Foundry, 2026-11-05)

Layer	What	Where
Foundry Data Health	4 checks: build status + primary key on `ontology_signal_signals` and `clean_faers_cases_current`	Dataset → **Health** tab
Computed checks	21 checks appended every build: keys, row-count drops, freshness, AI grounded rate, cache hits, reporter-type blanks	`Data/monitoring/monitoring_checks`
Drift replay	Weekly and per-quarter series rebuilt from source dates; last 4 weeks vs the 4 before	`Data/monitoring/monitoring_drift`

The first runs raised two warnings, and both are real OBSERVED. (1) FAERS reports with a blank *reporter type* (who sent it: doctor, consumer, lawyer…) rose from 34% in 2026 Q1 to 72% in Q2. That matters to the officer because a report from a doctor and one from a lawyer weigh differently, and she can no longer tell them apart for most Q2 reports (DQ38). (2) Recalls reported for our drugs in the last 4 weeks were 10 against 3 the 4 weeks before.

PLANNED	Schedules (so freshness checks fire on their own), a Health page in Workshop, and Data Health checks on recalls, DailyMed and the AI dataset.

What to click

1. Dataset → **Health** → **Add checks** → pick *Build status* or *Primary key*; write why in the note.
2. Monitoring results: dataset `monitoring_checks` → SQL console, filter on the latest `run_at`.

Gate: Chapters 4 to 13: every part can be found and checked TESTED

Required evidence: For each Foundry part: a name or path in our project, a picture or diagram, and the clicks.

Result: Met for all ten parts. Screens were captured on 5 November 2026; counts come from `numbers.json`.

If it fails: If a name has changed in Foundry, the mirror README in `foundry/` is the place to correct first.

14

Glossary and numbers

Term	Meaning in plain words
2×2 table (a, b, c, d)	Report counts for one drug and one reaction: a = drug + reaction, b = drug + other reactions, c = other drugs + reaction, d = the rest (chapter 2).
Action	A form that writes to the Ontology. Ours creates a Triage Decision.
Active recall	An openFDA recall with status Ongoing.
AIP	Palantir's Artificial Intelligence Platform: language models usable inside Foundry.
Baseline gate / gate	A gate is a written check that must pass before the next build step. The baseline gate is the 5 November 2026 rebuild whose counts are in `numbers.json`.
Bucket	Our keyword group for a reaction term (clinical, efficacy, abuse/overdose, product use, pregnancy, social/legal, administrative), a stand-in for licensed MedDRA groupings (D016 v3).
Build	One run of a transform; it writes new transactions to its output datasets.
Chi-squared (χ^2)	A test of whether a 2×2 table differs from what chance would give; Evans needs at least 4.
Code Repository	A git repository inside Foundry, holding transforms.
Data Health	Foundry's built-in checks and alerts on a dataset.
Data Lineage	The graph of which dataset feeds which, with build status.
Dataset	A versioned table (or set of files) in Foundry.
DailyMed / SPL	The US National Library of Medicine site for drug labels; SPL (Structured Product Labeling) is their XML format.
Egress policy	Permission for Foundry to send traffic to one outside host.
evaluation_id	A fingerprint (hash) of the inputs behind a signal; a triage decision stores it, so it records exactly what the officer saw.
Evans rule	A common screening rule for a signal: at least 3 reports, PRR at least 2, chi-squared at least 4.

Term	Meaning in plain words
F1	A score from 0 to 1 that balances missed items and wrong items; ours (0.97) covers a 35-item answer key only.
FAERS	The FDA Adverse Event Reporting System: spontaneous reports of suspected side effects.
GLP-1 drug	A newer diabetes and weight-loss medicine (semaglutide, liraglutide, dulaglutide, tirzepatide, exenatide).
Incremental transform	A transform that reads its own previous output and only processes new input.
Lightweight transform	A transform that runs on one machine (we use polars) instead of Spark.
Mass report	A FAERS report listing 30 or more drugs; kept out of the main count (D023 v2).
Medallion	The usual names bronze / silver / gold for the raw / clean / curated layers.
MedDRA preferred term (PT)	The standard medical word FAERS uses for a reaction, e.g. *Optic ischaemic neuropathy*. MedDRA's groupings need a licence we do not have.
NAION	Non-arteritic anterior ischaemic optic neuropathy: sudden vision loss from poor blood flow to the optic nerve. Our top semaglutide signal.
NDC	National Drug Code: the US code for a drug package.
Object / object type	One real-world thing (a drug) / the kind of thing (Drug), backed by a dataset.
Object Explorer	Foundry's search and browse tool for objects and links.
Ontology	Foundry's layer of objects, links and Actions on top of datasets.
openFDA	FDA's public API; we use its recall, Drugs@FDA and NDC endpoints.
PRR	Proportional reporting ratio: how much more often a reaction is reported with this drug than with all other drugs.
Project / folder	The shared space with permissions, and its folders.
Ranked	A signal in the review queue: not in label, clinical bucket, under half from mass reports.
Reference label	The one label per product we compare against: the application owner's label for that route.
RID	Resource identifier: a resource's permanent ID in Foundry.
RxNorm / RxCUI	The US standard drug vocabulary / its concept id. Our drug key is the ingredient RxCUI.
Signal	A drug and reaction that meets the Evans rule and is not found in any reference label (or only through a medium-band synonym: *possibly in label*). A lead to review, not proof.
Source	A Data Connection record for one outside system, used by code.
Suspect role	How a report names a drug: primary suspect, secondary suspect, interacting, or concomitant (not suspected).
Transaction	One write to a dataset: snapshot (replace), append, update or delete.
Transform	A Python function that reads datasets and writes datasets.
Workshop	Foundry's app builder: modules, pages, widgets, variables and events.

Figures quoted from project records (not in numbers.json)

These come from measured project records, not from the baseline-gate capture. Each is cited where it appears.

Figure	Source
NAION c = 128, d = 763,308 (other drugs)	Diagram 32 v2 (prints the `ontology_ae_stats` row)
9 top-level folders	foundry/README.md (resource map); ACTIVITY-LOG
930 raw recall rows = 900 distinct recalls (one row per recall per drug named)	foundry/README.md raw table
SQL console about 1.5 s; repository SQL panel no answer in 90 s	DECISIONS learnings, 2026-11-04
Lightweight builds about 1 min vs about 4 min for Spark	DECISIONS learnings, 2026-11-04
700 MB FAERS download; VS Code auto-commit for about 30 minutes	DECISIONS learnings, 2026-11-04; foundry/README.md
MSSM Signal: 25 properties; 22 shown in the property list; one paste vs about 60 clicks	screens/README.md (Figure 4); foundry/ontology/README.md; DECISIONS learnings, 2026-11-04
AIP access test 2.9 s	docs/10-delivery/2026-11-04-foundry-access-results.md
Answer key of 35 items (Boxed Warning + Contraindications, 2 labels)	eval/ai-eval-v1.md
5 random ranked signals traced	docs/11-handin/baseline-gate.md

Numbers used in this report

Every number in the table below is looked up from `docs/11-handin/numbers.json` at build time. Source: Foundry SQL (repo SQL panel) after the baseline-gate rebuild of the whole downstream chain: raw_dailymed_spl → clean_labels → clean_label_product_map → ontology_objects → ai_label_extraction → ontology_signals → ontology_changes → monitoring. Build durations from the Foundry Builds page. Query: docs/11-handin/baseline-gate.md. Captured: 2026-11-05T07:50Z. Figures quoted from other project records (decision log, data-issues register, optimization log) name that record where they appear.

Key	Value
`ai.d015_target_f1`	0.85
`ai.from_cache_on_rebuild`	9
`ai.gold_f1`	0.97
`ai.gold_recall_strict`	35/35
`ai.grounded`	448
`ai.model_calls_on_rebuild`	0
`ai.pilot_labels`	2
`ai.reactions`	448
`ai.sections`	9

Key	Value
app.labels_searched_naion	3
app.pages	4
app.triage_decisions	2
app.workshop_version	v0.9.0
changes.last_30_days	26
changes.snapshot_diff_after_rebuild	0
changes.total	2,898
faers.cases_current	792,346
labels.total	1,321
monitoring.checks_per_build	21
monitoring.data_health_checks	4
monitoring.faers_age_days	130
monitoring.recalls_last4w	10
monitoring.recalls_prev4w	3
monitoring.reporter_type_blank_q1	0.343
monitoring.reporter_type_blank_q2	0.721
optimization.O1.after_calls	0
optimization.O1.after_tokens	0
optimization.O1.before_calls	9
optimization.O1.before_tokens	70,776
optimization.O2.full_rebuild_s	32
optimization.O2.incremental_add_q2_s	88
raw.dailymed_labels	1,321
raw.drugsfda_rows	663
raw.ndc_rows	1,824
raw.openfda_requests_approx	50
raw.recall_rows	930
raw.recalls	900
recalls.active	126
recalls.total	900
scope.drugs_formulary	13

Key	Value
scope.drugs_with_signals	12
scope.glp1	5
scope.opioids	8
signals.ae_stats	14,921
signals.naion.a	394
signals.naion.all_reports_drug	20,892
signals.naion.chi2_yates	10653.8
signals.naion.prr	112.5
signals.naion.reaction	Optic ischaemic neuropathy
signals.not_in_label	1,844
signals.possibly_in_label	108
signals.ranked	1,073
signals.ranked_history.step9	1,355
signals.total	1,952